5mevsim — Halısaha Yönetim Platformu

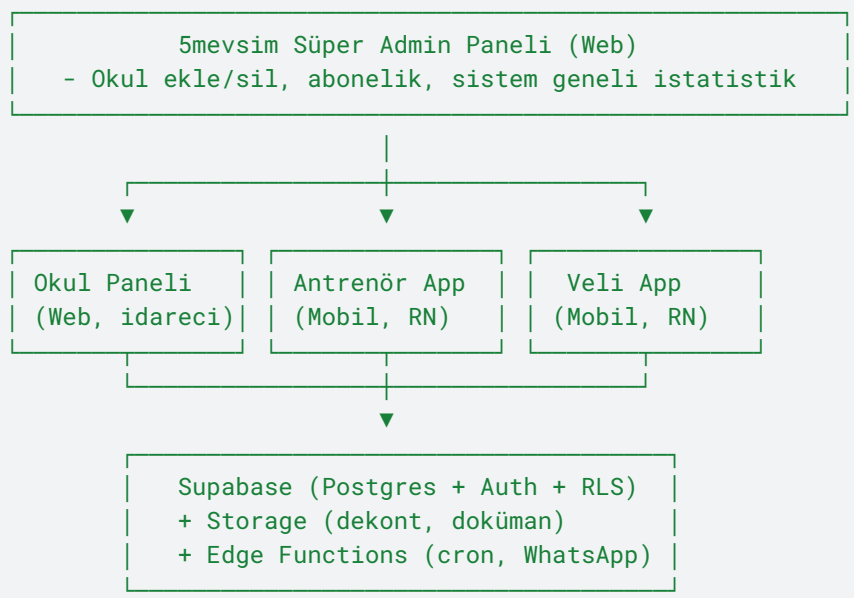
Veri Tabanı + Ekran Tasarımı (MVP)

> Hedef: Tek halısaha + futbol okulu için MVP. Mimari multi-tenant (SaaS) — sonradan başka okullara satılabilecek şekilde. Stack: Supabase (Postgres + Auth + Storage + RLS) + Next.js (web panel) + React Native (mobil). WhatsApp şimdilik kapalı; şablon altyapısı hazır olarak kalıyor.

---

1. Genel Mimari

None



4 rol:

- `super_admin` — sen, 5mevsim sahibi
- `school_admin` — her okulun idarecisi (giriş web panel)
- `coach` — antrenör (giriş mobil app, sadece yoklama + kendi gruplarını görür)
- `parent` — veli (giriş mobil app, sadece kendi çocuklarını)

Multi-tenant izolasyon: her tablo `school_id` taşır, Postgres RLS politikaları hangi kullanıcının hangi okulun verisini gördüğünü kilitler.

2. Veritabanı Şeması

2.1 Tenant ve Yetki

```sql
-- Her halısaha okulu = bir tenant
create table schools (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  slug text unique not null,                      -- 5mevsim.com/kapaklisitespor
  logo_url text,
  contact_phone text,
  address text,
  default_aidat_amount numeric(10,2) default 2500,
  default_uniform_amount numeric(10,2) default 2500,
  subscription_status text default 'trial'
    check (subscription_status in ('trial','active','suspended','cancelled')),
  subscription_starts_at timestamptz,
  subscription_ends_at timestamptz,
  created_at timestamptz default now()
);

-- Auth user başına 1 profile (Supabase auth.users 1-1)
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  school_id uuid references schools(id) on delete cascade,   -- super_admin için
NULL
  role text not null
    check (role in ('super_admin','school_admin','coach','parent')),
  full_name text not null,
  phone text not null,
  email text,
  is_active boolean default true,
  created_at timestamptz default now()
);
create index on profiles(school_id);
create index on profiles(phone);
```

2.2 Öğrenci ve Veli

```sql
create table students (
  id uuid primary key default gen_random_uuid(),
```

```
  school_id uuid references schools(id) on delete cascade not null,
  full_name text not null,
  birth_date date,
  gender text check (gender in ('M','F')),
  blood_type text,
  notes text,
  enrollment_date date default current_date,
  status text default 'active'
    check (status in ('active','passive','trial','left')),
  health_doc_url text,                          -- sağlık beyanı PDF
  consent_doc_url text,                         -- veli izin formu
  has_received_uniform boolean default false,   -- forma + eşofman alındı mı
  uniform_received_at date,
  created_at timestamptz default now()
);
create index on students(school_id);
create index on students(school_id, status);

-- Veli ↔ öğrenci (M-N: kardeşler tek veli, çift veli durumu)
create table parent_student (
  parent_profile_id uuid references profiles(id) on delete cascade,
  student_id uuid references students(id) on delete cascade,
  relation text default 'parent',               -- mother/father/guardian
  is_primary boolean default true,
  created_at timestamptz default now(),
  primary key (parent_profile_id, student_id)
);
create index on parent_student(student_id);
```

2.3 Personel (Antrenör + Ek)

SQL

```
create table staff (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  profile_id uuid references profiles(id),      -- antrenör için dolu, "abi" için
NULL olabilir
  full_name text not null,
  staff_type text not null
    check (staff_type in ('coach','assistant','admin','other')),
  monthly_salary numeric(10,2) default 0,
  hire_date date default current_date,
  is_active boolean default true,
  notes text,
  created_at timestamptz default now()
);
```

```
create index on staff(school_id, is_active);

-- Aylık maaş ödeme kayıtları
create table staff_salary_payments (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  staff_id uuid references staff(id) on delete cascade not null,
  period_year int not null,
  period_month int not null check (period_month between 1 and 12),
  amount numeric(10,2) not null,
  paid_at timestamptz default now(),
  payment_method text check (payment_method in ('cash','transfer','other')),
  notes text,
  unique(staff_id, period_year, period_month)
);
```

2.4 Grup ve Program

SQL

```
create table groups (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  name text not null,                          -- "U11 Salı-Perşembe 17:00"
  age_min int,
  age_max int,
  capacity int default 20,
  primary_coach_id uuid references staff(id),
  monthly_fee numeric(10,2) default 2500,      -- okul varsayılanını ezebilir
  is_active boolean default true,
  created_at timestamptz default now()
);
create index on groups(school_id, is_active);

-- Haftalık tekrar eden program slotları
-- Örn. grup haftada 2 gün ise burada 2 satır olur
create table group_schedule_slots (
  id uuid primary key default gen_random_uuid(),
  group_id uuid references groups(id) on delete cascade not null,
  day_of_week int not null check (day_of_week between 0 and 6), -- 0=Pazar
  start_time time not null,
  end_time time not null,
  created_at timestamptz default now()
);
create index on group_schedule_slots(group_id);

-- Öğrenci-grup atamaları (geçmişi de tutar)
```

```
create table student_groups (
  id uuid primary key default gen_random_uuid(),
  student_id uuid references students(id) on delete cascade not null,
  group_id uuid references groups(id) on delete cascade not null,
  joined_at date default current_date,
  left_at date,
  is_active boolean default true
);
create unique index on student_groups(student_id, group_id) where is_active;
```

2.5 Yoklama

SQL

```
-- Gerçekleşmiş veya planlanmış antrenman oturumu
-- Cron job her gün ertesi günün oturumlarını üretir (group_schedule_slots'tan)
create table attendance_sessions (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  group_id uuid references groups(id) on delete cascade not null,
  session_date date not null,
  start_time time not null,
  end_time time not null,
  status text default 'scheduled'
    check (status in ('scheduled','completed','cancelled')),
  cancellation_reason text,                    -- 'rain','holiday','other'
  cancelled_by uuid references profiles(id),
  cancelled_at timestamptz,
  taken_by uuid references profiles(id),       -- yoklamayı alan antrenör
  taken_at timestamptz,
  notes text,
  unique(group_id, session_date, start_time)
);
create index on attendance_sessions(school_id, session_date);

-- Öğrenci başına yoklama satırı
create table attendance_records (
  id uuid primary key default gen_random_uuid(),
  session_id uuid references attendance_sessions(id) on delete cascade not null,
  student_id uuid references students(id) on delete cascade not null,
  status text not null
    check (status in ('present','absent','excused','late')),
  -- excused: önceden veli izin bildirmiş
  -- absent : haber yok, gelmedi
  -- late   : geldi ama geç
  recorded_at timestamptz default now(),
  unique(session_id, student_id)
```

```
);

-- Veli mobilden bildirir (önceden); idareci/antrenör görür ve yoklamada excused
işler
create table leave_requests (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  student_id uuid references students(id) on delete cascade not null,
  parent_profile_id uuid references profiles(id),
  session_date date not null,
  group_id uuid references groups(id),
  reason text,
  status text default 'submitted'
    check (status in ('submitted','seen','applied')),
  created_at timestamptz default now()
);
create index on leave_requests(school_id, session_date);
```

2.6 Aidat ve Ödemeler

SQL

```
-- Aylık aidat faturaları (her ayın 1'inde cron job ile üretilir)
create table aidat_invoices (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  student_id uuid references students(id) on delete cascade not null,
  period_year int not null,
  period_month int not null check (period_month between 1 and 12),
  amount numeric(10,2) not null,                -- normalde 2500, ilk ay yarım
kayıt için 1250
  is_prorated boolean default false,            -- yarım aidat mı (ilk ay)
  due_date date not null,                       -- ayın 1'i
  paid_amount numeric(10,2) default 0,
  status text default 'pending'
    check (status in ('pending','partial','paid','waived')),
  notes text,
  created_at timestamptz default now(),
  unique(student_id, period_year, period_month)
);
create index on aidat_invoices(school_id, status);
create index on aidat_invoices(student_id, period_year, period_month);

-- Forma + eşofman tek seferlik fatura (öğrenci kayıt olunca veya manuel açılır)
create table equipment_invoices (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
```

```
  student_id uuid references students(id) on delete cascade not null,
  description text default 'Forma + Eşofman',
  amount numeric(10,2) not null,
  paid_amount numeric(10,2) default 0,
  status text default 'pending'
    check (status in ('pending','partial','paid','waived')),
  delivered boolean default false,
  delivered_at date,
  created_at timestamptz default now()
);
create index on equipment_invoices(school_id, status);

-- Tahsilat – bir aidat veya forma faturasına yapılan ödeme
create table payments (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  student_id uuid references students(id) on delete cascade not null,
  invoice_type text not null check (invoice_type in ('aidat','equipment')),
  invoice_id uuid not null,
  amount numeric(10,2) not null,
  payment_method text not null
    check (payment_method in ('cash','transfer','card','other')),
  payment_date date default current_date,
  receipt_url text,                              -- WhatsApp'tan gelen dekont
(Storage)
  received_by uuid references profiles(id),    -- ödemeyi kaydeden idareci
  notes text,
  created_at timestamptz default now()
);
create index on payments(school_id, payment_date);
create index on payments(invoice_type, invoice_id);
```

Not: `aidat_invoices.paid_amount` ve `status` alanları, ilgili `payments` satırları eklendiğinde trigger ile otomatik güncellenir (örnek trigger aşağıda — uygulamada ekleriz).

2.7 Başvuru / Deneme Antrenmanı (CRM)

SQL

```
create table applications (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  applicant_full_name text not null,            -- çocuğun adı
  birth_date date,
  parent_full_name text not null,
  parent_phone text not null,
```

```
  source text,                                  -- 'instagram','arkadaş','google'
  status text not null default 'new'
    check (status in (
      'new','contacted','trial_scheduled','trialed',
      'enrolled','rejected','no_show'
    )),
  trial_session_date date,
  trial_group_id uuid references groups(id),
  notes text,
  enrolled_student_id uuid references students(id),
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);
create index on applications(school_id, status);

create table application_status_history (
  id uuid primary key default gen_random_uuid(),
  application_id uuid references applications(id) on delete cascade,
  from_status text,
  to_status text,
  note text,
  changed_by uuid references profiles(id),
  changed_at timestamptz default now()
);
```

2.8 Giderler

SQL

```
create table expenses (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  category text not null
    check (category in ('rent','utility','equipment','salary','tax','other')),
  description text,
  amount numeric(10,2) not null,
  expense_date date default current_date,
  paid_to text,
  receipt_url text,
  created_by uuid references profiles(id),
  created_at timestamptz default now()
);
create index on expenses(school_id, expense_date);
```

2.9 Bildirim

```sql
create table notification_templates (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade,  -- NULL = platform ortak
  code text not null,                          -- 'aidat_reminder_5d'
  channel text not null check (channel in ('whatsapp','sms','push','in_app')),
  body_template text not null,                 -- "{{student_name}} için aidat 5 gün kaldı..."
  is_active boolean default true,
  created_at timestamptz default now(),
  unique(school_id, code, channel)
);

create table notification_log (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id) on delete cascade not null,
  recipient_profile_id uuid references profiles(id),
  recipient_phone text,
  channel text not null,
  template_code text,
  body_rendered text,
  status text default 'queued'
    check (status in ('queued','sent','delivered','failed')),
  error_message text,
  related_entity_type text,                    -- 'aidat_invoice','attendance'
  related_entity_id uuid,
  sent_at timestamptz,
  created_at timestamptz default now()
);
create index on notification_log(school_id, created_at desc);
```

2.10 Audit Log

```sql
create table audit_log (
  id uuid primary key default gen_random_uuid(),
  school_id uuid references schools(id),
  actor_profile_id uuid references profiles(id),
  action text not null,                        -- 'create_student','update_payment'
  entity_type text,
  entity_id uuid,
  diff jsonb,
```

```
  created_at timestamptz default now()
);
create index on audit_log(school_id, created_at desc);
```

2.11 RLS Politika Stratejisi (Özet)

SQL

```sql
-- Yardımcı fonksiyonlar
create or replace function current_school_id() returns uuid
language sql stable as $$
  select school_id from profiles where id = auth.uid()
$$;

create or replace function current_role() returns text
language sql stable as $$
  select role from profiles where id = auth.uid()
$$;

-- Tüm school_id'li tablolarda RLS aç
alter table students enable row level security;

-- 1) Süper admin her şeyi görür
create policy "super_admin_all" on students
  for all using (current_role() = 'super_admin');

-- 2) Okul idarecisi/antrenör kendi okulunu görür
create policy "school_users_select" on students
  for select using (
    school_id = current_school_id()
    and current_role() in ('school_admin','coach')
  );

-- 3) İdareci her şeyi yazar/günceller
create policy "school_admin_write" on students
  for all using (
    school_id = current_school_id()
    and current_role() = 'school_admin'
  );

-- 4) Veli sadece kendi çocuğunu görür
create policy "parent_select_own" on students
  for select using (
    current_role() = 'parent'
    and exists (
      select 1 from parent_student ps
      where ps.student_id = students.id
```

```
        and ps.parent_profile_id = auth.uid()
    )
  );
```

Aynı kalıp diğer tablolar için tekrarlanır. **Antrenör rolü çok dar tutulur** — sadece yoklama alabilsin, finansal/idari hiçbir şeye erişmesin:

Tablo	Antrenör Hakkı
`students`	sadece kendi gruplarındaki öğrencileri SELECT, yalnızca ad+foto görebilir (telefon/veli/borç gizli — view ile ayrı bir `student_basic_view` çıkar)
`groups`, `group_schedule_slots`	sadece kendi gruplarını SELECT
`attendance_sessions`	kendi gruplarının oturumlarını SELECT + UPDATE (status/cancelled)
`attendance_records`	kendi gruplarının oturumlarına INSERT/UPDATE
`leave_requests`	kendi gruplarının taleplerini SELECT
`aidat_invoices`, `equipment_invoices`, `payments`	**erişim yok**
`staff`, `staff_salary_payments`, `expenses`	**erişim yok**
`applications`, `parent_student`, `audit_log`	**erişim yok**

```SQL
-- Antrenör için kısıtlı öğrenci görünümü (telefon/veli yok)
create view student_basic_view as
  select id, school_id, full_name, birth_date, gender
  from students;

-- Antrenör attendance_records politikası örneği
create policy "coach_attendance_write" on attendance_records
  for all using (
```

```
    current_role() = 'coach'
    and exists (
      select 1 from attendance_sessions s
      join groups g on g.id = s.group_id
      where s.id = attendance_records.session_id
        and g.primary_coach_id in (
          select id from staff where profile_id = auth.uid()
        )
    )
  );
```

2.12 Trigger ve Cron İşleri

- **Aidat oto-üretim (ay başı):** ayın 1'i 00:05'te Supabase Edge Function — aktif öğrencilerin tümüne ay aidat invoice'u açar (`amount = monthly_fee`, `is_prorated = false`).
- **İlk ay yarım aidat (kayıt anında):** öğrenci kaydı oluşturulurken trigger devreye girer:
  - Kayıt tarihi `enrollment_date` o ayın faturasını oluşturur
  - `amount = monthly_fee / 2`, `is_prorated = true`
  - Sonraki ay başlarda cron normal tam aidatı açmaya devam eder
  - Mantık: ay içinde hangi gün kayıt olursa olsun, ilk ay her zaman yarım. Sonraki aylar tam.
- **Payment → invoice trigger:** `payments` tablosuna insert olunca `invoice_type/invoice_id`'ye göre `aidat_invoices` veya `equipment_invoices` `paid_amount` ve `status` günceller.
- **Yoklama oturumu üretimi:** her gece ertesi günün `attendance_sessions`'larını `group_schedule_slots`'tan üretir.
- **Aidat hatırlatması:** her gün 09:00 — bir sonraki ayın 1'ine 5 gün kalan ve borcu olan velilere bildirim kuyruğa atılır (WhatsApp kapalıyken sadece `notification_log`'a 'queued' olarak düşer; aktif olunca otomatik gönderilir).

---

3. Web Yönetim Paneli — Ekran Tasarımı

> Stack önerisi: Next.js 14 App Router + Tailwind + shadcn/ui + Supabase JS client.
> Sol sidebar + üstte breadcrumb + sağ ana içerik.

3.1 Layout

- **Sol sidebar (sabit):** Logo / Anasayfa / Öğrenciler / Veliler / Personel / Gruplar / Program / Yoklama / Aidat & Ödeme / Başvurular / Giderler / Raporlar / Bildirimler / Ayarlar / Çıkış.
- **Üst bar:** okul adı (SaaS müşterisi olduğunda burada okul ismi seçici), kullanıcı menüsü.

- **Süper admin bambaşka bir layout:** sol sidebar = Okullar / Abonelikler / Sistem / Bildirim Şablonları.

3.2 Anasayfa (Dashboard)

Üst sıra — KPI kartları:

- Toplam aktif öğrenci (`students` count where status='active')
- Bu ayki toplanan aidat (`payments` sum where invoice_type='aidat' & ay)
- Bekleyen aidat borcu (`aidat_invoices` sum amount−paid_amount where status≠'paid')
- Bekleyen başvuru (`applications` count where status in 'new','contacted')
- Tahsilat oranı yüzdesi (bu yıl içinde paid / total)
- Bugün devamsız (gerçek + henüz alınmamış oturumlardan)

İkinci sıra:

- Bugünün programı (saat saat liste — `group_schedule_slots` join `groups`)
- Son 5 ödeme (`payments` order by created_at desc)
- Devamsızlık trendi (son 30 gün) — bar chart
- Yıllık gelir / gider (line chart) — `payments` vs `expenses`

Üçüncü sıra:

- Grup dağılımı donut (öğrenci sayısı / grup)
- En çok devamsızlık yapan 5 öğrenci

3.3 Öğrenciler

Liste sayfası:

- Filtre çubuğu: ad, telefon, durum (aktif/pasif/deneme), grup, yaş aralığı, forma alındı mı
- Tablo: ID, Ad Soyad, Doğum/Yaş, Veli, Telefon, Aktif Grup, Forma Durumu, Borç Toplamı, İşlemler (gör / düzenle / pasife al)
- Üstte: "Yeni Öğrenci Ekle", "Toplu Excel Import" (V1.1)

Öğrenci Detay (sekmeler):

1. **Genel** — ad, doğum, kan grubu, sağlık dosyası, veli izin formu (upload)
2. **Veli(ler)** — bağlı veli profilleri, ekle/çıkar, kardeş gösterimi
3. **Gruplar** — `student_groups` üzerinden mevcut + geçmiş atamalar; "Gruba Ekle" butonu
4. **Yoklama Geçmişi** — son 90 günün takvim görünümü (yeşil=geldi, kırmızı=gelmedi, sarı=izinli)

5. **Aidat** — yıl seçici → 12 aylık grid: ödendi/kısmi/bekliyor; her satırda "Ödeme Al"
6. **Forma/Eşofman** — durum (alındı / alınmadı), tutar, ödeme, teslim tarihi
7. **Ödeme Geçmişi** — `payments` listesi, dekont indir
8. **Notlar / Audit** — kim ne zaman ne yapmış

3.4 Veliler

- Liste: telefon, ad, çocuk sayısı, toplam borç
- Detay: profil + bağlı çocuklar + tüm borç özeti (her çocuk için açılır kart)
- "Mobil app girişi davet et" butonu (SMS OTP linki gönderir)

3.5 Personel (Antrenör + Ek)

- Liste: ad, tip (antrenör/asistan/diğer), aylık maaş, grupları, durum
- Yeni personel: ad, tip, telefon, maaş, antrenörse → mobil hesap aç (auth user yarat)
- Detay: kişisel + sorumlu olduğu gruplar + maaş ödeme geçmişi
- "Maaş Öde" butonu — `staff_salary_payments` ekler, aynı zamanda `expenses` (category=salary) kaydı atar

3.6 Gruplar

- Liste: ad, gün/saat slotları (özet), antrenör, kapasite/dolu, aktif öğrenci
- Yeni grup: ad, yaş aralığı, kapasite, antrenör, **çoklu slot ekle** (gün + saat)
- Detay (sekmeler):
  - **Bilgi** — ad, yaş, kapasite, ücret
  - **Program** — haftalık slotlar
  - **Öğrenciler** — kayıtlı liste, ekle/çıkar
  - **Yoklama Geçmişi** — bu grubun oturum listesi
  - **İptal Edilen Antrenmanlar** — log

3.7 Program (Haftalık Takvim)

- Tek ekran: Pzt–Paz × saat dilimi grid (Google Calendar gibi)
- Kartlar = group_schedule_slots; renk antrenöre veya gruba göre
- Slot tıklanınca → grup detay
- "Bu Haftayı İptal Et" / "Bugünü İptal Et" butonu (resmi tatil veya yağmur)

3.8 Yoklama

- **Bugün** sekmesi: bugünün tüm oturumları (saat sırası), her satırda durum (alındı / bekliyor / iptal)
- **Geçmiş** sekmesi: tarih + grup filtresi
- Oturum detayı: öğrenci listesi + present/absent/excused/late toggle (idareci de tabletten alabilir)
- "İptal Et" — sebep (yağmur/tatil/diğer) + opsiyonel toplu bildirim

3.9 Aidat & Ödeme — En Kritik Ekran

Üst kısım — **Borç Özeti Kartları:**

- Toplam aidat borcu / Toplam forma borcu / Bu ay tahsil edilen / Genel toplam borç

Filtreler: yıl, grup, durum (borçlu/temiz), öğrenci adı.

Aidat Tab — Ana Tablo (piSPOR'daki gibi grid): | ID | Öğrenci | Grup | Aylık | Oca | Şub | Mar | Nis | May | Haz | Tem | Ağu | Eyl | Eki | Kas | Ara | Toplam Borç | Aksiyon |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 95 | Ahmet karaaslan | U11 17:00 | 2.500 | ✓ | ✓ | ✓ | ✓ | × | × | – | – | – | – | – | – | 5.000 | [Ödeme Al] |

İşaretler: ✓ ödendi / ⚠ kısmi / × ödenmedi / – henüz fatura yok (gelecek ay).

Forma Tab: alındı/alınmadı, ödendi/ödenmedi, teslim tarihi.

Ödeme Al Modali:

- Ödenecek fatura(lar) seç → tutar otomatik
- Ödeme yöntemi (nakit/havale/kart)
- Dekont yükle (Storage)
- Not
- Kaydet → `payments` insert → trigger faturayı günceller

Excel/PDF Export ve Yazdır butonları sağ üstte.

3.10 Başvurular (CRM — Kanban)

Sütunlar: **Yeni** → **Arandı** → **Deneme Atandı** → **Denemeye Geldi** → **Kayıt Oldu / Reddedildi / Gelmedi**

- Kart: çocuk adı, yaş, veli telefonu, kaynak, atanan deneme tarihi
- Kart sürükle-bırak ile durum değişir → `application_status_history` insert
- Tıklayınca detay modali: notlar, deneme atama (grup + tarih), "Öğrenciye Dönüştür" butonu (aktif öğrenci yarat)

3.11 Giderler

- Liste: tarih, kategori, açıklama, tutar, kim yarattı
- Yeni gider: kategori (kira/fatura/malzeme/maaş/diğer), tutar, makbuz upload
- Aylık özet kartları + kategori bazlı pasta grafik

- **Mekan Kirası** ayrı bir alt sekme (her ay otomatik hatırlatma)

3.12 Raporlar

- **Aylık Gelir-Gider Raporu** — tahsilat - giderler = net
- **Borç Raporu** — öğrenci × ay tablosu (Aidat ekranının export'u)
- **Devamsızlık Raporu** — öğrenci başına geldi/gelmedi yüzdesi (filtre: tarih aralığı, grup)
- **Tahsilat Raporu** — yöntem dağılımı (nakit / havale)
- **Grup Doluluk Raporu** — kapasite / kayıtlı öğrenci

3.13 Bildirimler

- **Log:** kime, ne zaman, hangi şablon, durum (queued/sent/failed)
- **Şablonlar:** kod, kanal, gövde (değişken: `{{full_name}}`, `{{amount}}`, `{{period}}`)
- **Manuel Toplu Bildirim** (V1.5): bir veya çoklu veliye custom mesaj — WhatsApp gelince aktif
- WhatsApp henüz kurulu değil iken: tüm mesajlar `queued` kalır, kuruldukça otomatik gönderilir

3.14 Ayarlar

- Okul bilgileri (logo, telefon, adres)
- Varsayılan aidat tutarı / forma tutarı
- Antrenman gün/saatleri (genel takvim default'u)
- İdareci kullanıcı yönetimi (yeni admin ekle)
- Bildirim ayarları (hangi otomatik mesajlar açık)
- Veri yedekleme / dışa aktarma

3.15 Süper Admin Paneli (5mevsim ana paneli, ayrı login)

- **Okullar Listesi** — her okul kart/satır: isim, abonelik durumu, öğrenci sayısı, son giriş
- **Yeni Okul Ekle** — okul kayıt + ilk admin user
- **Abonelik** — başlangıç, bitiş, durum güncelle
- **Sistem Geneli İstatistik** — kaç okul, kaç öğrenci, toplam ciro
- **Bildirim Şablonu Yönetimi** — platform varsayılanları
- **Audit / Sistem Logları**

---

4. Mobil Uygulama — Ekran Tasarımı

Tek React Native projesi, login sonrası role'e göre ya veli ya antrenör akışına yönlendirilir. SMS OTP girişi (Supabase Auth phone provider).

4.1 Veli Uygulaması

Login akışı:

1. Telefon numarası → SMS OTP
2. OTP doğrulama
3. Profile yoksa: ad soyad iste, oluştur (idareci öğrencileri telefonla bağladığı için profil otomatik bulunur)
4. Anasayfa'ya yönlen

Anasayfa:

- Üstte selam + okul logo
- Çocuk kartları (kardeş varsa hepsi listelenir): ad, foto/avatar, durum rozetleri (borç var/yok, sıradaki antrenman)
- KPI: toplam borç (büyük rakam, kırmızı/yeşil)
- "Bu Hafta" şeridi: önümüzdeki 7 günde antrenmanlar mini liste

Çocuk Detay (üstte sekme bar):

1. **Program** — haftalık takvim, sıradaki antrenman vurgulu
2. **Yoklama** — takvim görünümü (yeşil/kırmızı/sarı noktalar) + "Bu Ay" özet (geldi: 6, gelmedi: 1, izinli: 1)
3. **Borç & Ödeme**
    - Aidat aylık liste (✓ ödendi / × açık / ⚠ kısmi)
    - Forma/eşofman durumu
    - Toplam borç
    - **"Dekont Gönder"** butonu — fotoğraf çek/seç → idareciye gider, idareci panelden onaylayınca ödeme görünür
4. **İzin Bildir**
    - Tarih seç (sadece kendi çocuğunun antrenman günleri)
    - Sebep (opsiyonel)
    - Gönder → `leave_requests` insert; antrenör yoklamada görür

Bildirimler tab: geçmiş bildirim log (in-app)

Profil tab: kendi bilgileri, çıkış

4.2 Antrenör Uygulaması (Çok Dar — Sadece Yoklama)

Antrenör fiilen tek bir şey yapar: kendi gruplarının yoklamasını alır. Finansal bilgi, başka grup, veli telefonu, borç durumu vs. görmez.

Login akışı: telefon + SMS OTP (idareci antrenör kaydı açtığında profile oluşur)

Anasayfa:

- Bugün eğiteceği gruplar listesi (saat sırasıyla)
- Her grup kartı: saat, grup adı, öğrenci sayısı, "Yoklama Al" butonu (oturum başlangıç saatinden 30 dk önce aktif)
- Altta küçük link: "Geçmiş yoklamalarım"

Yoklama Ekranı (tek kritik ekran):

- Üstte grup adı + tarih + saat
- Öğrenci listesi: sadece **ad + foto/avatar** (telefon, veli, borç vs. yok)
- 4 buton her satırda: ✓ Geldi / ✗ Gelmedi / ⏰ Geç / 🛌 İzinli
    - "İzinli" zaten `leave_requests`'te varsa otomatik işaretli ve kilitli
- Hepsini "Geldi" işaretle butonu (default)
- "Yoklamayı Tamamla" → `attendance_session.status='completed'`, `taken_by` antrenör

Antrenman İptal:

- Sebep seç (yağmur / tatil / diğer)
- Onay → idareciye bildirim, `attendance_session.status='cancelled'`

Geçmiş Yoklamalar: sadece kendi grupları + tarih filtresi, salt okunur

Profil: kendi adı, telefonu (maaş bilgisi GÖRÜNMEZ), çıkış

> Antrenör web panele girememeli. Web login akışında role kontrolü yapılır, `coach` rolü mobil app'e yönlendirilir.

---

5. MVP Sprint Önerisi (8 hafta)

Hafta	Konu	Çıktı
1	Altyapı + Auth	Supabase kurulum, multi-tenant şema, RLS, profile flow, Next.js + RN iskelet

Hafta	Konu	Çıktı
2	Öğrenci + Veli + Personel	CRUD ekranları (web), öğrenci detay sekmeleri
3	Grup + Program	Grup CRUD, slot yönetimi, haftalık takvim ekranı
4	Yoklama (web + antrenör mobil)	Cron oturum üretimi, antrenör mobil yoklama, idareci tarafı
5	Aidat + Ödeme	Otomatik aidat üretimi, ödeme alma, aidat grid'i, forma faturası
6	Veli mobil	Login, çocuk listesi, program, borç görüntüleme, dekont gönderme, izin bildirme
7	Başvuru CRM + Giderler + Raporlar	Kanban, gider modülü, ana raporlar
8	Süper admin + Bildirim altyapısı + QA	5mevsim paneli, notification queue (WhatsApp olmadan), test, dağıtım

WhatsApp Business onaylandıktan sonra → 1 haftalık ek sprint: webhook bağlantısı, şablon onayları, otomatik gönderim devreye alma.

---

6. Açık Bırakılanlar / Sonraya

- Saha kiralama modülü (V2)
- Online ödeme entegrasyonu (V2)
- Antrenman fotoğraf/video paylaşımı (V2)
- Etkinlik / maç / turnuva (V2)
- WhatsApp iki yönlü bot (V2)
- Excel toplu öğrenci import (V1.1)
- Manuel toplu bildirim ekranı (V1.5)
- Marka tescil + üretim ortamı geçişi (Türkiye sunucu)

---

7. Sonraki Adım Önerisi

Bu doküman onayından sonra:

1. **Supabase projesi açıp şemayı kurmak** — yukarıdaki SQL'i tek migration olarak yatırmak
2. **Wireframe çizmek** — özellikle 3.9 (Aidat ekranı) ve 4.1 Veli "Borç & Ödeme" ekranları için Figma/eskiz
3. **Repo açmak** — monorepo (apps/web, apps/mobile, packages/shared)

İlk somut iş olarak Supabase migration dosyasını ve örnek seed verisi (1 okul, 5 antrenör, 10 öğrenci, 3 grup) hazırlayabilirim — projeyi 1 saatte ayağa kaldırabilirsin.